



作业三：对称密码算法的软件实现

网安李瑞佳 学号：202300460078

网安岳智宇 学号：202300460074

网安张金浩 学号：202300460059

密码薛俊东 学号：202300460087

2026 年 7 月 26 日

目录

1	实验目的	2
2	实验环境	2
3	实验原理	2
3.1	SM4 分组密码基本原理	2
3.2	密钥扩展原理	3
3.3	Baseline 实现原理	4
3.4	T-table 优化原理	4
3.5	Shuffle 多分组并行优化原理	5
3.6	CTR 工作模式原理	6
4	系统设计与实现	6
4.1	系统总体结构与调用关系	7
4.2	统一上下文与公共数据处理	8
4.3	Baseline 实现步骤	8
4.4	T-table 优化实现步骤	10
4.5	Shuffle 优化实现步骤	11
4.6	CTR 与 CTR T-table 工作模式实现步骤	12
4.7	性能测试入口设计	13
5	测试过程	13
5.1	编译过程	14
5.2	正确性测试	14
5.3	性能测试	14
6	实验结果分析	15
6.1	第一次运行结果	15
6.2	第二次运行结果	15
6.3	第三次运行结果	16
6.4	三次性能结果汇总	17
6.5	正确性结果分析	18
6.6	核心算法优化效果分析	18
6.7	CTR 工作模式优化效果分析	18
6.8	综合分析	19
7	实验总结	19
7.1	完成情况总结	19
7.2	与作业要求的对应关系	19
7.3	不足与改进方向	20
7.4	实验反思	20

1 实验目的

本实验题目为“作业 3: 对称密码算法的软件实现”。实验选择 SM4 分组密码作为优化对象, 从基础软件实现出发, 完成 Baseline、T-table 优化、Shuffle 批量并行优化、普通 CTR 工作模式以及 CTR T-table 优化工作模式的软件实现与测试。实验目的如下:

1. 理解 SM4 分组密码的轮函数、密钥扩展、加密解密流程和大端数据处理方式。
2. 在 Baseline 实现基础上, 掌握 T-table 预计算优化和 Shuffle 多块并行优化两种典型软件优化方法。
3. 实现 CTR 工作模式, 并将 T-table 优化后端接入 CTR 密钥流生成过程, 理解分组密码核心优化对工作模式吞吐率的影响。
4. 通过正确性测试、交叉验证和性能测试, 分析不同实现的吞吐量、加速比和适用场景。

本实验覆盖了作业要求中“从基本实现出发优化对称密码软件执行效率”的内容, 并在 T-table、Shuffle、最新指令集三类方法中完成了前两类方法; 工作模式方面选择 CTR 模式, 并实现了 Baseline 后端 CTR 与 T-table 后端 CTR 两种版本。

2 实验环境

- 操作系统: Windows 环境, PowerShell 终端。
- 编译器: GCC/MinGW-w64。
- 编程语言: C 语言, 采用 C99 标准。
- 编译选项: `-O3 -march=native -Wall -Wextra -std=c99 -Iinclude`。
- 测试数据规模: 性能测试中使用 1MB 随机数据, 即 65536 个 SM4 分组。

其中, `-O3`用于启用高级编译优化, `-march=native`用于让编译器根据本机处理器生成较适合的指令序列, `-Wall -Wextra`用于增强编译期检查。

3 实验原理

本实验选择 SM4 分组密码作为对称密码算法的软件实现与优化对象。SM4 是我国商用密码标准中的典型分组密码算法, 分组长度为 128 bit, 密钥长度为 128 bit, 加密过程由 32 轮迭代组成。实验从直观的 Baseline 实现出发, 在保持算法功能等价的前提下, 分别实现 T-table 查表优化和 Shuffle 多分组并行优化, 并进一步实现 CTR 工作模式, 使 SM4 能够处理任意长度的数据流。

3.1 SM4 分组密码基本原理

SM4 算法以 128 bit 明文分组和 128 bit 主密钥作为输入。一个 128 bit 明文分组首先被划分为 4 个 32 bit 字:

$$(X_0, X_1, X_2, X_3)$$

算法共执行 32 轮迭代。第 i 轮的轮函数可表示为:

$$X_{i+4} = X_i \oplus T(X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)$$

其中, rk_i 为第 i 轮轮密钥, $\oplus$ 表示按位异或, T 为 SM4 加密轮函数中的合成变换。32 轮完成后, 算法并不是直接输出 $(X_{32}, X_{33}, X_{34}, X_{35})$, 而是按照反序输出:

$$(Y_0, Y_1, Y_2, Y_3) = (X_{35}, X_{34}, X_{33}, X_{32})$$

这也对应代码中最后使用 `PUT_UINT32_BE` 按 x_3, x_2, x_1, x_0 顺序写回输出缓冲区的实现方式。SM4 轮函数中的 T 变换由非线性变换 τ 和线性变换 L 组成:

$$T(A) = L(\tau(A))$$

设 32 bit 输入 A 由 4 个字节组成:

$$A = (a_0, a_1, a_2, a_3)$$

非线性变换 τ 对每个字节分别查 S 盒:

$$\tau(A) = (Sbox(a_0), Sbox(a_1), Sbox(a_2), Sbox(a_3))$$

线性变换 L 定义为:

$$L(B) = B \oplus ROTL(B, 2) \oplus ROTL(B, 10) \oplus ROTL(B, 18) \oplus ROTL(B, 24)$$

其中, $ROTL(B, n)$ 表示对 32 bit 字进行循环左移 n 位。可以看出, SM4 每一轮都需要进行字节级 S 盒替换、循环移位和异或运算。Baseline 实现直接按照上述数学定义完成非线性变换和线性扩散过程, 虽然没有进行针对硬件特性的优化, 但代码结构清晰, 能够准确反映 SM4 算法的基本执行流程, 因此适合作为正确性验证基准以及后续优化方案的性能对照对象。

3.2 密钥扩展原理

SM4 加密前需要由 128 bit 主密钥生成 32 个轮密钥。主密钥记为:

$$MK = (MK_0, MK_1, MK_2, MK_3)$$

首先与系统参数 FK 异或, 得到初始密钥状态:

$$K_i = MK_i \oplus FK_i, \quad i = 0, 1, 2, 3$$

随后使用固定参数 CK_i 生成轮密钥:

$$rk_i = K_i \oplus T'(K_{i+1} \oplus K_{i+2} \oplus K_{i+3} \oplus CK_i)$$

其中 T' 同样由 S 盒替换和线性变换组成, 但其线性变换 L' 与加密轮函数中的 L 不同:

$$L'(B) = B \oplus ROTL(B, 13) \oplus ROTL(B, 23)$$

本实验代码中, `sm4_baseline_set_key` 负责完成密钥扩展, 并将轮密钥正序保存到 `enc_rk` 中, 将轮密钥逆序保存到 `dec_rk` 中。SM4 解密过程与加密过程结构相同, 只需要按相反顺序使用轮密钥。因此, 解密函数可以复用同样的轮函数, 只是将 `enc_rk` 替换为 `dec_rk`。

3.3 Baseline 实现原理

Baseline 实现严格按照 SM4 标准流程逐轮计算。每个输入分组先按大端序读取为 4 个 32 bit 字, 然后执行 32 轮迭代。每一轮中, 程序先计算:

$$A = X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i$$

再对 A 的 4 个字节分别查 S 盒, 并执行线性变换 L , 最后与 X_i 异或得到新的状态字。

Baseline 的优点是实现直接、可读性强, 便于验证 SM4 算法逻辑是否正确。但从性能角度看, 每轮都需要执行 4 次 S 盒访问、多次移位、多次异或和状态更新。对于大量分组数据, 这些重复的字节拆分、拼接和循环移位会成为主要开销。因此, 后续优化的核心目标就是在保持轮函数等价的前提下, 减少每轮的计算量, 或者提高多个分组之间的并行度。

3.4 T-table 优化原理

T-table 优化是一种典型的时间换空间方法。SM4 轮函数中的 T 变换可以写为:

$$T(A) = L(\tau(A))$$

设:

$$A = (a_0, a_1, a_2, a_3)$$

则:

$$\tau(A) = (Sbox(a_0), Sbox(a_1), Sbox(a_2), Sbox(a_3))$$

由于 L 是线性变换, 满足异或可分解性, 因此可以将 4 个字节在不同位置上对最终 T 变换结果的贡献提前计算出来:

$$T(A) = T_0[a_0] \oplus T_1[a_1] \oplus T_2[a_2] \oplus T_3[a_3]$$

其中:

$$T_0[a] = L(Sbox(a) \ll 24)$$

$$T_1[a] = L(Sbox(a) \ll 16)$$

$$T_2[a] = L(Sbox(a) \ll 8)$$

$$T_3[a] = L(Sbox(a))$$

这样, 每轮原本需要执行的 S 盒查找、字节拼接和多次循环左移, 可以被替换为 4 次 32 bit 查表和 3 次异或。本实验中使用大小为 4×256 的 `uint32_t` 数组 `T_TABLE` 保存预计算结果, 总空间约为 4KB。初始化时程序遍历所有可能的字节值, 计算其经过 S 盒和线性变换后的贡献, 并分别存入 4 张表对应的位置。

运行时, T-table 版本的轮函数只需执行:

$$tt = T_0[(A \gg 24) \& 0xff] \oplus T_1[(A \gg 16) \& 0xff] \oplus T_2[(A \gg 8) \& 0xff] \oplus T_3[A \& 0xff]$$

再计算:

$$X_{i+4} = X_i \oplus tt$$

该方法减少了轮函数中的循环移位和字节重组操作, 因此通常能提升吞吐率。不过, T-table 优化也会引入更多表访问, 对缓存命中率有一定依赖。如果表数据能够稳定驻留在高速缓存中, 性能收益更明显; 如果缓存压力较大, 查表访问开销可能抵消部分优化效果。

3.5 Shuffle 多分组并行优化原理

Shuffle 优化的核心思想是改变数据组织方式, 将多个独立分组的相同状态字放在一起处理, 从而提高指令级并行度, 并为编译器自动向量化创造条件。SM4 单个分组内部的 32 轮迭代存在前后依赖关系, 即后一轮依赖前一轮产生的状态, 因此单个分组内部很难完全并行。但是, 不同明文分组之间互不依赖, 可以并行执行相同轮次的计算。

本实验的 Shuffle 实现一次处理 8 个 SM4 分组。程序将 8 个分组的从普通分组连续存储形式:

$$B_0 = (x_{0,0}, x_{0,1}, x_{0,2}, x_{0,3}), \quad B_1 = (x_{1,0}, x_{1,1}, x_{1,2}, x_{1,3}), \dots$$

重排为按状态字分类的数组形式:

$$x0[8], \quad x1[8], \quad x2[8], \quad x3[8]$$

也就是代码中的 `ShuffleState` 结构。这样, 在同一轮中, 8 个分组都会执行相同的计算:

$$t_j = x1[j] \oplus x2[j] \oplus x3[j] \oplus rk_i, \quad j = 0, 1, \dots, 7$$

然后分别计算:

$$tt_j = T(t_j)$$

最后更新:

$$tmp_j = x0[j] \oplus tt_j$$

$$x0[j] = x1[j], \quad x1[j] = x2[j], \quad x2[j] = x3[j], \quad x3[j] = tmp_j$$

这种方式没有改变 SM4 算法本身, 而是改变了多分组数据在内存和寄存器中的排列方式。其优势主要体现在三个方面: 第一, 8 个分组共享同一轮密钥, 减少了循环调度开销; 第二, 同类操作连续出现, 处理器流水线更容易保持高利用率; 第三, 在 `-O3 -march=native` 等编译优化条件下, 编译器更容易将数组循环转换为 SIMD 或其他并行指令序列。

本实验的 Shuffle 实现还结合了 T-table 思想, 在 `shuffle_T` 中使用预计算表完成 T 变换。因此它既减少了单轮计算量, 又通过 8 块批处理提升了多分组吞吐率。对于不足 8 个分组的尾部数据, 程序回退到 Baseline 单块加解密, 以保证任意分组数量都能正确处理。

3.6 CTR 工作模式原理

分组密码本身只能处理固定长度的数据块, 而实际应用中明文长度通常是任意的。CTR 模式将分组密码转换为类似流密码的工作方式, 利用分组密码加密连续变化的计数器块, 生成密钥流, 再与明文异或得到密文。

CTR 模式中, 每个输入块对应一个计数器块:

$$Ctr_i = IV \parallel counter_i$$

其中 IV 为初始化向量, $counter_i$ 为递增计数器。本实验代码中, 计数器写入 16 字节 Counter Block 的末尾 8 字节, 并采用大端方式存放。每个计数器块经过 SM4 加密后得到 16 字节密钥流:

$$KS_i = E_K(Ctr_i)$$

加密过程为:

$$C_i = P_i \oplus KS_i$$

解密过程为:

$$P_i = C_i \oplus KS_i$$

由于异或运算满足自反性:

$$(P_i \oplus KS_i) \oplus KS_i = P_i$$

所以 CTR 模式的加密和解密可以使用同一个核心函数。本实验中的 `sm4_ctr_encrypt` 和 `sm4_ctr_decrypt` 都调用 `sm4_ctr_crypt`, 区别只在于输入数据分别是明文或密文。

CTR 模式具有几个重要特点。首先, 它不需要填充, 因为最后一个分组可以只取密钥流的前若干字节与剩余数据异或。其次, CTR 模式只使用分组密码的加密函数, 不需要调用分组密码解密函数。再次, 不同计数器块之间相互独立, 因此 CTR 模式天然适合并行优化: 可以预先构造多个 Counter Block, 再使用 T-table 或 Shuffle 版本的 SM4 批量生成密钥流。

本实验在 CTR 实现中保留了两种后端。普通 CTR 使用 `sm4_baseline_encrypt_block` 生成密钥流, 用于观察完整工作模式在 Baseline 核心上的开销; 优化 CTR 使用 `sm4_ttable_encrypt_block` 生成密钥流, 对应代码中的 `sm4_ctr_ttable_encrypt`, 用于验证 T-table 核心优化接入 CTR 后能否提高工作模式吞吐率。进一步改进时, 还可以一次构造 8 个连续 Counter Block 并调用 `sm4_shuffle_encrypt_blocks` 批量生成密钥流, 从而继续发挥 CTR 模式天然可并行的优势。

4 系统设计与实现

本实验的软件系统围绕 SM4 分组密码的软件实现、核心算法优化以及 CTR 工作模式优化三个层次展开。整体设计路线是: 先实现一个严格对应 SM4 标准流程的 Baseline 版本, 作为正确性和性能基准; 再在相同上下文和轮密钥接口上实现 T-table 查表优化与 Shuffle 多分组并行优化; 最后将 CTR 工作模式分别接入 Baseline 后端和 T-table 后端, 形成普通 CTR 与 CTR T-table 两种工作模式实现。这样既能满足题目中“从基本实现出发优化对称密码执行效率”的要求, 也能体现“基于加解密软件实现工作模式优化”的要求。

当前代码中,普通 CTR 调用`sm4_baseline_encrypt_block`生成密钥流;优化 CTR 调用`sm4_ttable_encrypt_block`生成密钥流,并通过`sm4_ctr_ttable_encrypt`对外提供接口。因此,报告中的 CTR 部分不只是模式封装,而是包含了工作模式后端替换带来的软件优化。

4.1 系统总体结构与调用关系

实验代码按照公共头文件、算法实现文件和测试程序三类组织,目录结构如下:

```
1 lab3/
2 |-- Makefile
3 |-- README.md
4 |-- include/
5 |   |-- sm4.h
6 |   |-- sm4_constants.h
7 |   |-- sm4_baseline.h
8 |   |-- sm4_ttable.h
9 |   |-- sm4_shuffle.h
10 |   |-- sm4_ctr.h
11 |-- src/
12 |   |-- sm4_baseline.c
13 |   |-- sm4_ttable.c
14 |   |-- sm4_shuffle.c
15 |   |-- sm4_ctr.c
16 |-- test/
17 |   |-- test_sm4.c
18 |   |-- test_perf.c
19 |   |-- test_crosscheck.c
20 |   |-- test_ttable_math.c
```

其中, `sm4_constants.h`保存 S 盒、系统参数FK、固定参数CK、循环左移函数以及大端序读写宏; `sm4.h`定义统一上下文结构和所有公开接口; `sm4_baseline.c`实现基础 SM4; `sm4_ttable.c`实现 T-table 优化; `sm4_shuffle.c`实现 8 分组批处理优化; `sm4_ctr.c`实现普通 CTR 和 CTR T-table 两种工作模式。

系统的主要调用关系可以概括为:

1. 正确性测试调用 Baseline、T-table 和 Shuffle 接口,验证加密后再解密能否恢复明文。
2. 性能测试先分别测试三种 SM4 核心后端,再测试普通 CTR 和 CTR T-table。
3. T-table 和 Shuffle 均复用 Baseline 密钥扩展,使不同实现之间的差异主要集中在轮函数和数据组织方式上。
4. CTR Mode 使用 Baseline 后端生成密钥流,CTR T-table Mode 使用 T-table 后端生成密钥流。

这种分层方式使报告后续的性能分析具有明确对象: Baseline、T-table、Shuffle 用于比较分组密码核心执行效率; CTR Mode 和 CTR T-table Mode 用于比较同一工作模式在不同后端下的整体吞吐量。

4.2 统一上下文与公共数据处理

所有 SM4 实现共用SM4_Ctx上下文结构。该结构定义在sm4.h中:

```
1 typedef struct {
2     uint32_t rk[SM4_ROUNDS];
3     uint32_t enc_rk[SM4_ROUNDS];
4     uint32_t dec_rk[SM4_ROUNDS];
5     uint8_t iv[SM4_BLOCK_SIZE];
6     uint64_t counter;
7 } SM4_Ctx;
```

rk保存密钥扩展得到的 32 个原始轮密钥; enc_rk保存加密时使用的正序轮密钥; dec_rk保存解密时使用的逆序轮密钥。由于 SM4 解密只需要反向使用加密轮密钥, 因此保存dec_rk可以让加密和解密共享同一轮函数结构。CTR 模式所需的iv和counter也放在同一上下文中, 便于在流式处理时维护状态。

SM4 标准按大端方式解释 32 bit 字, 因此公共头文件中定义了大端序读写宏:

```
1 #define GET_UINT32_BE(p) \
2     (((uint32_t)(p)[0] << 24) | ((uint32_t)(p)[1] << 16) | \
3     ((uint32_t)(p)[2] << 8) | ((uint32_t)(p)[3]))
4
5 #define PUT_UINT32_BE(p, v) \
6     do { \
7         (p)[0] = (uint8_t)((v) >> 24); \
8         (p)[1] = (uint8_t)((v) >> 16); \
9         (p)[2] = (uint8_t)((v) >> 8); \
10        (p)[3] = (uint8_t)(v); \
11    } while (0)
```

这两个宏保证了代码在不同处理器大小端环境下都按照 SM4 标准解释输入输出。Baseline、T-table、Shuffle 在读入分组和写回密文时都使用这组宏, 因此不同后端之间具有一致的数据格式。

4.3 Baseline 实现步骤

Baseline 实现位于src/sm4_baseline.c, 它是整个系统的基础版本。其核心目标是尽量直接地把 SM4 数学定义翻译为 C 代码, 因此适合作为正确性参照。

首先, 密钥扩展函数sm4_baseline_set_key将 16 字节主密钥按大端序读成 4 个 32 bit 字, 并与系统参数FK异或:

```
1 k[0] = GET_UINT32_BE(key) ^ FK[0];
2 k[1] = GET_UINT32_BE(key + 4) ^ FK[1];
3 k[2] = GET_UINT32_BE(key + 8) ^ FK[2];
4 k[3] = GET_UINT32_BE(key + 12) ^ FK[3];
```

随后执行 32 轮密钥扩展, 每轮使用CK[i]和密钥扩展变换sm4_t_prime_baseline生成一个轮密钥:

```
1 for (int i = 0; i < SM4_ROUNDS; i++) {
2     uint32_t t = k[1] ^ k[2] ^ k[3] ^ CK[i];
3     ctx->rk[i] = k[0] ^ sm4_t_prime_baseline(t);
4 }
```

```
5   k[0] = k[1];
6   k[1] = k[2];
7   k[2] = k[3];
8   k[3] = ctx->rk[i];
9 }
```

密钥扩展结束后, 程序同时构造加密轮密钥和解密轮密钥:

```
1  for (int i = 0; i < SM4_ROUNDS; i++) {
2      ctx->enc_rk[i] = ctx->rk[i];
3      ctx->dec_rk[i] = ctx->rk[SM4_ROUNDS - 1 - i];
4  }
```

Baseline 的轮函数 `sm4_t_baseline` 直接体现了 SM4 的 `Sbox` 替换和线性变换 `L`。代码先从 32 bit 输入中拆出 4 个字节, 分别查 `S` 盒, 再重新组合为 32 bit 字:

```
1  uint8_t b0 = (x >> 24) & 0xFF;
2  uint8_t b1 = (x >> 16) & 0xFF;
3  uint8_t b2 = (x >> 8) & 0xFF;
4  uint8_t b3 = x & 0xFF;
5
6  uint32_t y = ((uint32_t)SBOX[b0] << 24) |
7              ((uint32_t)SBOX[b1] << 16) |
8              ((uint32_t)SBOX[b2] << 8) |
9              (uint32_t)SBOX[b3];
```

随后执行线性扩散:

```
1  return y ^ ROTL32(y, 2) ^ ROTL32(y, 10) ^
2         ROTL32(y, 18) ^ ROTL32(y, 24);
```

单块加密函数 `sm4_baseline_encrypt_block` 按大端序读入 4 个状态字, 执行 32 轮迭代:

```
1  for (int i = 0; i < SM4_ROUNDS; i++) {
2      uint32_t t = x1 ^ x2 ^ x3 ^ ctx->enc_rk[i];
3      uint32_t tmp = x0 ^ sm4_t_baseline(t);
4      x0 = x1;
5      x1 = x2;
6      x2 = x3;
7      x3 = tmp;
8  }
```

最后按 SM4 规定反序输出:

```
1  PUT_UINT32_BE(out, x3);
2  PUT_UINT32_BE(out + 4, x2);
3  PUT_UINT32_BE(out + 8, x1);
4  PUT_UINT32_BE(out + 12, x0);
```

解密函数与加密函数结构相同, 只是使用 `ctx->dec_rk[i]`。因此, Baseline 实现清楚体现了 SM4 “同一轮函数、反序轮密钥解密” 的结构特点。

4.4 T-table 优化实现步骤

T-table 优化位于src/sm4_ttable.c。该模块的目标是减少每轮运行时对 S 盒替换、字节组合和循环左移的重复计算。代码中定义了 1024 个 32 bit 表项:

```
1 static uint32_t T_TABLE[1024];
2 static bool ttable_initialized = false;
```

1024 个表项可以理解为 4 张 256 项的表, 分别对应输入字节位于 32 bit 字的不同位置时对最终T变换的贡献。初始化函数只在第一次设置密钥时执行:

```
1 for (int i = 0; i < 256; i++) {
2     uint32_t s = (uint32_t)SBOX[i];
3     uint32_t ls = s ^ ROTL32(s, 2) ^ ROTL32(s, 10) ^
4                 ROTL32(s, 18) ^ ROTL32(s, 24);
5
6     T_TABLE[i] = ROTL32(ls, 24);
7     T_TABLE[256 + i] = ROTL32(ls, 16);
8     T_TABLE[512 + i] = ROTL32(ls, 8);
9     T_TABLE[768 + i] = ls;
10 }
```

这里ls表示一个 S 盒输出字节在最低字节位置时经过线性变换后的结果; 通过循环左移 24、16、8 位, 可以得到该字节分别位于最高字节、次高字节、次低字节时的贡献。由于 SM4 的线性变换由异或和循环移位组成, 满足按字节贡献异或合成的性质, 因此运行时可以直接查表合并:

```
1 uint32_t tt = T_TABLE[(t >> 24) & 0xFF] ^
2               T_TABLE[256 + ((t >> 16) & 0xFF)] ^
3               T_TABLE[512 + ((t >> 8) & 0xFF)] ^
4               T_TABLE[768 + (t & 0xFF)];
```

T-table 版本的单轮状态更新仍然保持 SM4 结构不变:

```
1 uint32_t tmp = x0 ^ tt;
2 x0 = x1;
3 x1 = x2;
4 x2 = x3;
5 x3 = tmp;
```

也就是说,T-table 优化只改变T变换的计算方式, 不改变 SM4 轮函数数学含义。代码中sm4_ttable_set_key复用 Baseline 密钥扩展:

```
1 void sm4_ttable_set_key(SM4_Ctx *ctx, const uint8_t *key) {
2     init_ttable();
3     sm4_baseline_set_key(ctx, key);
4 }
```

这种设计保证 T-table 版本和 Baseline 版本使用完全相同的轮密钥, 使性能差异主要来自查表优化本身。

4.5 Shuffle 优化实现步骤

Shuffle 优化位于src/sm4_shuffle.c。SM4 单个分组内部的 32 轮迭代具有串行依赖, 但多个分组之间互不依赖。因此, Shuffle 版本选择一次处理 8 个分组, 将同类状态字集中存放, 提高多分组处理吞吐率。批处理状态结构如下:

```
1 typedef struct {
2     uint32_t x0[8];
3     uint32_t x1[8];
4     uint32_t x2[8];
5     uint32_t x3[8];
6 } ShuffleState;
```

输入的 8 个分组原本按字节连续存储, 程序先通过bytes_to_state把它们重排为 4 组状态字数组:

```
1 for (int i = 0; i < 8; i++) {
2     state->x0[i] = GET_UINT32_BE(in[i]);
3     state->x1[i] = GET_UINT32_BE(in[i] + 4);
4     state->x2[i] = GET_UINT32_BE(in[i] + 8);
5     state->x3[i] = GET_UINT32_BE(in[i] + 12);
6 }
```

这种布局使同一轮中 8 个分组可以执行相同操作。单轮批处理函数shuffle_round分三步进行。第一步, 对 8 个分组分别计算轮函数输入:

```
1 for (int i = 0; i < 8; i++) {
2     t[i] = state->x1[i] ^ state->x2[i] ^ state->x3[i] ^ rk;
3 }
```

第二步, 对 8 个t[i]分别调用shuffle_T。该函数内部同样使用预计算表SHUFFLE_T, 因此 Shuffle 版本同时利用了 T-table 减少轮函数计算量的思想:

```
1 for (int i = 0; i < 8; i++) {
2     tt[i] = shuffle_T(t[i]);
3 }
```

第三步, 更新 8 个分组的狀態:

```
1 for (int i = 0; i < 8; i++) {
2     uint32_t tmp = state->x0[i] ^ tt[i];
3     state->x0[i] = state->x1[i];
4     state->x1[i] = state->x2[i];
5     state->x2[i] = state->x3[i];
6     state->x3[i] = tmp;
7 }
```

批量加密函数先计算完整的 8 块批次数量和尾部剩余数量:

```
1 size_t num_batches = num_blocks / 8;
2 size_t remaining = num_blocks % 8;
```

完整批次执行 32 轮shuffle_round; 不足 8 块的尾部数据回退到 Baseline 单块加密:


```
1 for (size_t i = 0; i < remaining; i++) {
2     sm4_baseline_encrypt_block(ctx,
3         rem_in + i * SM4_BLOCK_SIZE,
4         rem_out + i * SM4_BLOCK_SIZE);
5 }
```

这种尾部回退设计保证了接口可以处理任意分组数量, 而不要求输入长度必须是 8 个分组的整数倍。解密函数采用同样结构, 只是每轮使用 `ctx->dec_rk[round]`。

4.6 CTR 与 CTR T-table 工作模式实现步骤

CTR 模式位于 `src/sm4_ctr.c`。普通 CTR 初始化函数调用 Baseline 密钥扩展, 保存 IV 和初始计数器:

```
1 void sm4_ctr_init(SM4_Ctx *ctx, const uint8_t *key,
2     const uint8_t *iv, uint64_t counter) {
3     sm4_baseline_set_key(ctx, key);
4     memcpy(ctx->iv, iv, SM4_BLOCK_SIZE);
5     ctx->counter = counter;
6 }
```

核心加解密函数每次复制 IV 构造 Counter Block, 再把 64 bit 计数器写入末尾 8 字节:

```
1 memcpy(counter_block, ctx->iv, SM4_BLOCK_SIZE);
2 uint64_t ctr = ctx->counter;
3 for (int i = 7; i >= 0; i--) {
4     counter_block[SM4_BLOCK_SIZE - 8 + i] = (uint8_t)(ctr & 0xFF);
5     ctr >>= 8;
6 }
```

普通 CTR 使用 Baseline 后端生成密钥流:

```
1 sm4_baseline_encrypt_block(ctx, counter_block, keystream);
```

然后取当前剩余长度和 16 字节分组长度中的较小值作为 chunk, 支持最后不足一块的数据:

```
1 size_t remaining = length - processed;
2 size_t chunk = (remaining < SM4_BLOCK_SIZE) ?
3     remaining : SM4_BLOCK_SIZE;
```

密钥流与输入数据逐字节异或:

```
1 for (size_t i = 0; i < chunk; i++) {
2     out[processed + i] = in[processed + i] ^ keystream[i];
3 }
4 processed += chunk;
5 ctx->counter++;
```

CTR T-table 模式与普通 CTR 保持相同的 Counter Block 构造、异或和计数器递增流程, 但初始化阶段调用 T-table 密钥设置:

```
1 void sm4_ctr_ttable_init(SM4_Ctx *ctx, const uint8_t *key,
2     const uint8_t *iv, uint64_t counter) {
```

```
3     sm4_ttable_set_key(ctx, key);
4     memcpy(ctx->iv, iv, SM4_BLOCK_SIZE);
5     ctx->counter = counter;
6 }
```

在密钥流生成阶段, 优化版本将 Baseline 后端替换为 T-table 后端:

```
1 sm4_ttable_encrypt_block(ctx, counter_block, keystream);
```

因此, 两种 CTR 实现的区别非常明确: 计数器构造和异或逻辑完全相同, 性能差异主要来自底层分组加密后端。由于 CTR 模式加密和解密都只是与同一密钥流异或, 代码中 `sm4_ctr_encrypt` 和 `sm4_ctr_decrypt` 调用同一个 `sm4_ctr_crypt`, `sm4_ctr_ttable_encrypt` 和 `sm4_ctr_ttable_decrypt` 调用同一个 `sm4_ctr_ttable_crypt`。

4.7 性能测试入口设计

性能测试程序 `test_perf.c` 使用 1MB 随机数据作为输入。SM4 分组大小为 16 字节, 因此测试数据共包含 65536 个分组。测试程序依次执行五类测试:

1. Baseline: 循环调用 `sm4_baseline_encrypt_block`。
2. T-table: 循环调用 `sm4_ttable_encrypt_block`。
3. Shuffle: 调用 `sm4_shuffle_encrypt_blocks` 批量处理全部分组。
4. CTR Mode: 调用 `sm4_ctr_encrypt`, 底层使用 Baseline 后端。
5. CTR T-table Mode: 调用 `sm4_ctr_ttable_encrypt`, 底层使用 T-table 后端。

新增的 CTR T-table 性能测试代码如下:

```
1 sm4_ctr_ttable_init(&ctx, key, iv, 0);
2 start = get_time();
3 sm4_ctr_ttable_encrypt(&ctx, plain, cipher, DATA_SIZE);
4 end = get_time();
5 ctr_ttable_time = end - start;
6 print_result("CTR T-table Mode", ctr_ttable_time, DATA_SIZE);
7 printf(" Speedup vs CTR: %.2fx\n", ctr_time / ctr_ttable_time);
```

吞吐量计算公式为:

$$Throughput = \frac{DataSize}{Time \times 1024 \times 1024}$$

其中, T-table 和 Shuffle 的加速比相对 Baseline 计算; CTR T-table 额外输出相对普通 CTR 的加速比。这样既能评价 SM4 核心算法优化, 也能评价 CTR 工作模式接入优化后端后的整体效果。

5 测试过程

本实验测试过程包括编译测试、正确性测试和性能测试三部分。测试命令均在 `lab3` 目录下执行。

5.1 编译过程

首先执行:

```
1 mingw32-make
```

截图中可以看到, 编译器使用如下参数生成test_sm4和test_perf:

```
1 gcc -O3 -march=native -Wall -Wextra -std=c99 -linclude
```

其中-O3用于开启高级优化, -march=native允许编译器利用本机处理器特性, -Wall -Wextra用于增加警告检查。编译过程未出现错误, 说明新增的 CTR T-table 接口已经正确声明、实现并链接进测试程序。

5.2 正确性测试

正确性测试通过运行:

```
1 .\test_sm4.exe
```

完成。测试程序分别对 Baseline、T-table 和 Shuffle 执行加密再解密, 并用memcmp比较解密结果与原始明文是否一致。截图中三次运行均显示:

```
1 Testing SM4 basic encryption/decryption...
2   Baseline: PASS
3   T-table: PASS
4   Shuffle: PASS
5
6 All tests passed!
```

这说明基础实现和两种优化实现都能正确完成 SM4 加解密。由于 CTR 和 CTR T-table 都只是在不同后端上生成密钥流, 并使用异或完成加解密, 因此只要底层 SM4 后端正确, CTR 模式的正确性基础也成立。若进一步加强测试, 可以增加专门的 CTR 往返测试, 即分别调用sm4_ctr_encrypt/sm4_ctr_decrypt和sm4_ctr_ttable_encrypt/sm4_ctr_ttable_decrypt, 验证任意长度输入均能恢复原文。

5.3 性能测试

性能测试通过运行:

```
1 .\test_perf.exe
```

完成。测试数据规模为 1MB, 即 65536 个 SM4 分组。程序依次测试 Baseline、T-table、Shuffle、普通 CTR 以及 CTR T-table 五类实现。当前test_perf.c中新增的 CTR T-table 测试逻辑如下:

```
1 sm4_ctr_ttable_init(&ctx, key, iv, 0);
2 start = get_time();
3 sm4_ctr_ttable_encrypt(&ctx, plain, cipher, DATA_SIZE);
4 end = get_time();
5 ctr_ttable_time = end - start;
6 print_result("CTR T-table Mode", ctr_ttable_time, DATA_SIZE);
7 printf(" Speedup vs CTR: %.2fx\n", ctr_time / ctr_ttable_time);
8 printf(" Speedup vs Baseline: %.2fx\n", baseline_time / ctr_ttable_time);
```

因此,性能测试不仅比较了核心 SM4 优化方法,还直接比较了普通 CTR 和 T-table 优化 CTR 之间的吞吐率差异,更符合题目中“工作模式的软件优化实现”的要求。

6 实验结果分析

三张新版实验截图均包含编译、`test_perf.exe`性能测试和`test_sm4.exe`正确性测试。与旧结果相比,性能测试新增了CTR T-table Mode,可以直接观察 CTR 工作模式接入 T-table 后端后的加速效果。

6.1 第一次运行结果

第一次运行结果如图 1 所示。

```
PS J:\大三下\网络安全创新创业实践\实验三\lab3> mingw32-make
gcc -O3 -march=native -Wall -Wextra -std=c99 -Iinclude -o test_sm4 src/sm4_baseline.o src/sm4_ttable.o src/sm4_shuffle.o src/sm4_ctr.o test/test_sm4.c -lm
gcc -O3 -march=native -Wall -Wextra -std=c99 -Iinclude -o test_perf src/sm4_baseline.o src/sm4_ttable.o src/sm4_shuffle.o src/sm4_ctr.o test/test_perf.c -lm
PS J:\大三下\网络安全创新创业实践\实验三\lab3> .\test_perf.exe

=== SM4 Performance Test (1MB data) ===

Baseline           : 0.0109 s,    91.99 MB/s
T-table            : 0.0085 s,   118.15 MB/s
Speedup: 1.28x
Shuffle            : 0.0065 s,   154.90 MB/s
Speedup: 1.68x
CTR Mode           : 0.0118 s,    84.90 MB/s
CTR T-table Mode   : 0.0084 s,   118.93 MB/s
Speedup vs CTR: 1.40x
Speedup vs Baseline: 1.29x
PS J:\大三下\网络安全创新创业实践\实验三\lab3> .\test_sm4.exe
Testing SM4 basic encryption/decryption...
Baseline: PASS
T-table: PASS
Shuffle: PASS

All tests passed!
PS J:\大三下\网络安全创新创业实践\实验三\lab3> 
```

图 1: 第一次运行结果: CTR T-table 模式加入后的性能测试与正确性测试

第一次测试中, Baseline 耗时 0.0109s, 吞吐率为 91.99MB/s; T-table 耗时 0.0085s, 吞吐率为 118.15MB/s, 加速比为 1.28x; Shuffle 耗时 0.0065s, 吞吐率为 154.90MB/s, 加速比为 1.68x; 普通 CTR 耗时 0.0118s, 吞吐率为 84.90MB/s; CTR T-table 耗时 0.0084s, 吞吐率为 118.93MB/s。CTR T-table 相对普通 CTR 加速 1.40x, 相对 Baseline 加速 1.29x。

该结果说明, T-table 后端不仅能提升单块 SM4 加密速度,也能在 CTR 工作模式中提升密钥流生成效率。

6.2 第二次运行结果

第二次运行结果如图 2 所示。

```

PS J:\大三下\网络安全创新创业实践\实验三\lab3> mingw32-make
gcc -O3 -march=native -Wall -Wextra -std=c99 -Iinclude -o test_sm4 src/sm4_baseline.o sr
c/sm4_ttable.o src/sm4_shuffle.o src/sm4_ctr.o test/test_sm4.c -lm
gcc -O3 -march=native -Wall -Wextra -std=c99 -Iinclude -o test_perf src/sm4_baseline.o s
rc/sm4_ttable.o src/sm4_shuffle.o src/sm4_ctr.o test/test_perf.c -lm
PS J:\大三下\网络安全创新创业实践\实验三\lab3> .\test_perf.exe

=== SM4 Performance Test (1MB data) ===

Baseline      : 0.0106 s, 94.75 MB/s
T-table       : 0.0087 s, 115.19 MB/s
Speedup: 1.22x
Shuffle       : 0.0083 s, 120.83 MB/s
Speedup: 1.28x
CTR Mode      : 0.0113 s, 88.53 MB/s
CTR T-table Mode : 0.0105 s, 94.81 MB/s
Speedup vs CTR: 1.07x
Speedup vs Baseline: 1.00x
PS J:\大三下\网络安全创新创业实践\实验三\lab3> .\test_sm4.exe
Testing SM4 basic encryption/decryption...
Baseline: PASS
T-table: PASS
Shuffle: PASS

All tests passed!
PS J:\大三下\网络安全创新创业实践\实验三\lab3>

```

图 2: 第二次运行结果: 普通 CTR 与 CTR T-table 对比

第二次测试中, Baseline 耗时 0.0106s, 吞吐率为 94.75MB/s; T-table 耗时 0.0087s, 吞吐率为 115.19MB/s, 加速比为 1.22x; Shuffle 耗时 0.0083s, 吞吐率为 120.83MB/s, 加速比为 1.28x; 普通 CTR 耗时 0.0113s, 吞吐率为 88.53MB/s; CTR T-table 耗时 0.0105s, 吞吐率为 94.81MB/s。CTR T-table 相对普通 CTR 加速 1.07x, 相对 Baseline 约为 1.00x。

这一组结果中, CTR T-table 提升幅度较小, 说明工作模式优化效果会受到运行时系统负载、缓存状态以及计时波动影响。但即使在该次运行中, CTR T-table 仍然快于普通 CTR。

6.3 第三次运行结果

第三次运行结果如图 3所示。

```

● PS J:\大三下\网络安全创新创业实践\实验三\lab3> mingw32-make
gcc -O3 -march=native -Wall -Wextra -std=c99 -Iinclude -o test_sm4 src/sm4_baseline.o sr
c/sm4_ttable.o src/sm4_shuffle.o src/sm4_ctr.o test/test_sm4.c -lm
gcc -O3 -march=native -Wall -Wextra -std=c99 -Iinclude -o test_perf src/sm4_baseline.o s
rc/sm4_ttable.o src/sm4_shuffle.o src/sm4_ctr.o test/test_perf.c -lm
● PS J:\大三下\网络安全创新创业实践\实验三\lab3> .\test_perf.exe

=== SM4 Performance Test (1MB data) ===

Baseline           : 0.0110 s,    90.97 MB/s
T-table            : 0.0104 s,    96.40 MB/s
Speedup: 1.06x
Shuffle            : 0.0077 s,   129.13 MB/s
Speedup: 1.42x
CTR Mode           : 0.0130 s,    76.78 MB/s
CTR T-table Mode   : 0.0091 s,   109.85 MB/s
Speedup vs CTR: 1.43x
Speedup vs Baseline: 1.21x
● PS J:\大三下\网络安全创新创业实践\实验三\lab3> .\test_sm4.exe
Testing SM4 basic encryption/decryption...
Baseline: PASS
T-table: PASS
Shuffle: PASS

All tests passed!
❖ PS J:\大三下\网络安全创新创业实践\实验三\lab3>

```

图 3: 第三次运行结果: T-table 后端对 CTR 模式的加速效果

第三次测试中, Baseline 耗时 0.0110s, 吞吐率为 90.97MB/s; T-table 耗时 0.0104s, 吞吐率为 96.40MB/s, 加速比为 1.06x; Shuffle 耗时 0.0077s, 吞吐率为 129.13MB/s, 加速比为 1.42x; 普通 CTR 耗时 0.0130s, 吞吐率为 76.78MB/s; CTR T-table 耗时 0.0091s, 吞吐率为 109.85MB/s。CTR T-table 相对普通 CTR 加速 1.43x, 相对 Baseline 加速 1.21x。

该结果再次说明, 当 CTR 底层密钥流生成函数由 Baseline 替换为 T-table 后, 完整 CTR 模式的吞吐率可以明显提升。

6.4 三次性能结果汇总

三次性能测试数据汇总如表 1 所示。

表 1: 三次运行性能测试结果汇总

实现方式	第 1 次耗时/s	第 1 次吞吐率/MB/s	第 2 次耗时/s	第 2 次吞吐率/MB/s	第 3 次耗时/s	第 3 次吞吐率/MB/s
Baseline	0.0109	91.99	0.0106	94.75	0.0110	90.97
T-table	0.0085	118.15	0.0087	115.19	0.0104	96.40
Shuffle	0.0065	154.90	0.0083	120.83	0.0077	129.13
CTR Mode	0.0118	84.90	0.0113	88.53	0.0130	76.78
CTR T-table Mode	0.0084	118.93	0.0105	94.81	0.0091	109.85

三次平均结果如表 2 所示。

表 2: 三次测试平均性能

实现方式	平均耗时/s	平均吞吐率/MB/s	平均加速比	说明
Baseline	0.0108	92.57	1.00x	基准实现
T-table	0.0092	109.91	1.19x	核心轮函数查表优化
Shuffle	0.0075	134.95	1.46x	8 分组批处理优化
CTR Mode	0.0120	83.40	0.90x	Baseline 后端 CTR
CTR T-table Mode	0.0093	107.86	1.30x vs CTR	T-table 后端 CTR

从平均结果看, T-table 核心加密相对 Baseline 平均加速约 1.19x, Shuffle 平均加速约 1.46x。普通 CTR 由于需要构造 Counter Block、维护计数器并执行逐字节异或, 平均吞吐率低于 Baseline; CTR T-table 平均吞吐率为 107.86MB/s, 相对普通 CTR 平均加速约 1.30x, 说明工作模式接入优化后端后确实改善了整体性能。

6.5 正确性结果分析

三次截图中的 `test_sm4.exe` 均显示 Baseline、T-table 和 Shuffle 为 PASS, 说明三类 SM4 核心实现均能完成加密后再解密的正确往返。T-table 改变了 τ 变换的计算方式, Shuffle 改变了多分组状态组织方式, 二者仍然通过测试, 说明优化没有破坏 SM4 算法语义。

需要说明的是, 当前 `test_sm4.exe` 主要验证核心分组密码实现。新增的 CTR T-table 模式已经在性能测试中被调用并成功运行; 若继续完善测试体系, 可以增加专门的 CTR 正确性测试, 比较普通 CTR 与 CTR T-table 在相同密钥、IV 和计数器下生成的密文是否一致, 并验证解密是否恢复原文。

6.6 核心算法优化效果分析

T-table 优化通过预计算 S 盒和线性变换组合结果, 将每轮复杂的字节替换、循环移位和异或扩散转化为 4 次查表和异或。三次测试中 T-table 均快于 Baseline, 说明该优化方向有效。不过 T-table 依赖缓存访问, 性能波动较明显, 三次吞吐率从 96.40MB/s 到 118.15MB/s 不等。

Shuffle 优化利用多分组之间的独立性, 一次处理 8 个分组。它的优势主要来自数据组织方式改变和循环调度开销降低。三次测试中 Shuffle 均快于 Baseline, 但第二次和第三次吞吐率低于第一次, 说明批处理优化同样会受到系统负载、缓存状态以及自动优化效果波动的影响。整体而言, Shuffle 仍是本实验中平均吞吐率最高的核心 SM4 实现。

6.7 CTR 工作模式优化效果分析

普通 CTR 模式底层调用 `sm4_baseline_encrypt_block` 生成密钥流, 因此除了 SM4 加密本身, 还需要额外执行 IV 复制、计数器写入、密钥流异或和计数器递增。三次测试中普通 CTR 吞吐率分别为 84.90MB/s、88.53MB/s 和 76.78MB/s, 均低于或接近 Baseline, 符合完整工作模式存在额外开销的预期。

CTR T-table 模式将密钥流生成函数替换为 `sm4_ttable_encrypt_block`。三次测试中, 其吞吐率分别为 118.93MB/s、94.81MB/s 和 109.85MB/s, 相对普通 CTR 加速比分别为 1.40x、1.07x 和 1.43x。由此可见, 在 CTR 模式中替换优化分组密码后端可以有效降低密钥流生成成本, 使完整工作模式性能得到提升。

这一部分说明, 本实验不仅完成了 CTR 模式的软件实现, 还将 T-table 优化后端接入 CTR 密钥流生成过程, 更紧扣题目中“基于加解密软件实现, 做 CTR/GCM/XTS 工作模式的软件优化实现”的要求。

6.8 综合分析

综合三次测试结果, 可以得到以下结论:

1. Baseline 实现正确稳定, 可作为性能基准。
2. T-table 优化能够减少 SM4 轮函数运行时计算量, 平均加速约 1.19x。
3. Shuffle 优化通过 8 分组批处理提升吞吐量, 平均加速约 1.46x。
4. 普通 CTR 完整实现存在计数器维护和异或开销, 吞吐量低于单纯分组加密基准。
5. CTR T-table 模式相对普通 CTR 平均加速约 1.30x, 说明工作模式层面的后端优化有效。
6. 三次正确性测试全部 PASS, 说明核心优化没有破坏加解密正确性。

从题目要求看, 本实验选择 SM4 作为对称密码算法, 覆盖了 T-table 和 Shuffle 两种优化方法, 并在 CTR 工作模式中实现了 Baseline 后端和 T-table 优化后端。因此, 实验结果能够同时体现“核心分组密码优化”和“工作模式优化”两个层面的效果。

7 实验总结

本实验围绕“对称密码算法的软件实现”这一作业要求展开, 选择 SM4 作为实现对象, 完成了基础加解密、T-table 优化、Shuffle 优化、普通 CTR 模式以及 CTR T-table 优化模式。相较于最初只实现普通 CTR 的版本, 当前实验进一步将 T-table 优化接入 CTR 工作模式, 使工作模式部分也具有明确的优化对比。

7.1 完成情况总结

从算法实现看, Baseline 版本完成了 SM4 密钥扩展、32 轮加密、反序输出和逆序轮密钥解密流程, 是整个实验的正确性基准。从优化方法看, T-table 版本用查表减少轮函数计算量, Shuffle 版本用 8 分组批处理提高并行度, 覆盖了题目要求中 T-table、Shuffle、最新指令集三类方法中的两类。从工作模式看, 实验实现了 CTR 模式, 并进一步实现了 `sm4_ctr_ttable_encrypt`, 使 CTR 密钥流生成阶段能够使用 T-table 后端。

从测试结果看, 三次正确性测试均显示 Baseline、T-table 和 Shuffle 为 PASS。性能测试中, T-table 和 Shuffle 均快于 Baseline; 普通 CTR 由于包含工作模式额外开销, 性能低于单纯分组加密; CTR T-table 相对普通 CTR 平均加速约 1.30x, 说明工作模式优化取得了实际效果。

7.2 与作业要求的对应关系

本实验与题目要求对应关系如下:

1. 题目要求选择 SM4/AES/GIFT/TWINE 之一, 本实验选择 SM4。

2. 题目要求从基本实现出发优化软件执行效率, 本实验以 Baseline 为基础, 增加 T-table 和 Shuffle 优化。
3. 题目要求至少覆盖 T-table、Shuffle、最新指令集中的两种方法, 本实验覆盖 T-table 和 Shuffle。
4. 题目要求选择 CTR/GCM/XTS 之一进行工作模式优化, 本实验选择 CTR, 并实现 Baseline 后端 CTR 与 T-table 后端 CTR。
5. 题目要求体现执行效率优化, 本实验通过 1MB 数据测试给出耗时、吞吐率和加速比。

因此, 当前实验报告和代码已经较完整地覆盖了作业要求。

7.3 不足与改进方向

本实验仍有改进空间。首先, CTR T-table 已经实现, 但还可以继续实现 CTR Shuffle 版本: 一次构造 8 个连续 Counter Block, 再调用 `sm4_shuffle_encrypt_blocks` 批量生成密钥流, 这会更充分发挥 CTR 天然可并行的特点。其次, 本实验没有显式使用最新指令集或 SM4 专用硬件指令; 虽然 `-march=native` 会让编译器生成适合本机的指令, 但它不等同于手写 SIMD 或专用密码指令优化。后续若环境支持, 可以加入显式 SIMD 版本, 与 T-table、Shuffle 进行更完整比较。

此外, T-table 虽然提升了速度, 但查表访问可能带来缓存侧信道风险。在真实密码工程应用中, 性能不是唯一目标, 还需要考虑常数时间实现、缓存访问模式和侧信道防护。因此, T-table 适合作为本实验中的软件优化方法, 但在高安全场景中需要谨慎使用。

7.4 实验反思

这次实验让我更明显地感受到, 密码算法实现有两个层次: 一个是把算法“写对”, 另一个是把算法“写快”。Baseline 解决的是第一个问题, T-table 和 Shuffle 解决的是第二个问题。特别是 Shuffle 优化说明, 很多性能提升并不是改变密码算法本身, 而是改变数据组织方式, 让处理器更容易连续处理相似操作。

加入 CTR T-table 之后, 我也更清楚地意识到, 工作模式不是简单地在分组加密外面套一层循环。普通 CTR 要构造 Counter Block、维护计数器、生成密钥流并逐字节异或, 因此完整模式的速度可能低于单块加密基准。只有把优化后的分组密码后端真正接入 CTR, 才能说工作模式也得到了优化。

最后, 三次测试结果并不完全一致, 也提醒我性能实验不能只看一次运行。缓存状态、系统负载、计时误差都会影响吞吐率。比较可靠的做法是多次运行, 观察趋势, 再解释波动原因。对我来说, 这次实验不仅是在写 SM4 代码, 也是在学习如何用更工程化、更审慎的方式评价密码软件实现。